

```

# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

/kaggle/input/distil_bert/keras/distil_bert_base_en_uncased/2/
config.json
/kaggle/input/distil_bert/keras/distil_bert_base_en_uncased/2/tokenize
r.json
/kaggle/input/distil_bert/keras/distil_bert_base_en_uncased/2/metadata
.json
/kaggle/input/distil_bert/keras/distil_bert_base_en_uncased/2/model.we
ights.h5
/kaggle/input/distil_bert/keras/distil_bert_base_en_uncased/2/assets/
tokenizer/vocabulary.txt
/kaggle/input/nlp-getting-started/sample_submission.csv
/kaggle/input/nlp-getting-started/train.csv
/kaggle/input/nlp-getting-started/test.csv

#Extracting training data from csv to a pandas dataframe
df_train= pd.read_csv("/kaggle/input/nlp-getting-started/train.csv")
df_train.head()

```

	id	keyword	location	text
0	1	NaN	NaN	Our Deeds are the Reason of this #earthquake M...
1	4	NaN	NaN	Forest fire near La Ronge Sask. Canada
2	5	NaN	NaN	All residents asked to 'shelter in place'

```

are ...
3  6      NaN      NaN  13,000 people receive #wildfires evacuation
or...
4  7      NaN      NaN  Just got sent this photo from Ruby #Alaska
as ...

```

```

    target
0        1
1        1
2        1
3        1
4        1

```

#Extracting testing data from csv to a pandas dataframe

```

df_test= pd.read_csv("/kaggle/input/nlp-getting-started/test.csv")
df_test.head()

```

```

    id keyword location
text
0  0      NaN      NaN      Just happened a terrible car
crash
1  2      NaN      NaN  Heard about #earthquake is different cities,
s...
2  3      NaN      NaN  there is a forest fire at spot pond, geese
are...
3  9      NaN      NaN      Apocalypse lighting. #Spokane
#wildfires
4  11      NaN      NaN      Typhoon Soudelor kills 28 in China and
Taiwan

```

Taking a quick look at our data to understand what a non-disaster and a disaster tweet look like

```

from sklearn.model_selection import train_test_split
# Values and targets
X = df_train["text"]
y = df_train["target"]

# Split into train and validation sets
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=.2,
random_state=42)

#extracting the "text" column from the df_test DataFrame and assigning
it to the variable X_test
X_test = df_test["text"]

```

Setting up a DistilBERT model for sentiment analysis (2 classes) on text data. Here's the breakdown:

- **Preprocessing:** Importing libraries and creating a DistilBertPreprocessor to handle tokenization and padding of text inputs for a maximum length of 160 characters (tweets).
- **Model Building:** Building a DistilBertClassifier using a pre-trained DistilBERT base model with the preprocessor I created and specifying two output classes (likely positive and negative sentiment).
- **Training Setup:** Compiling the model with an appropriate loss function (Sparse Categorical Crossentropy), optimizer (Adam with a learning rate of 1e-5), and accuracy metric. Finally, I'm training the model on my training data (X_train and y_train) for 2 epochs, evaluating on the validation set (X_val and y_val) using a batch size of 32 for training efficiency.

```
#import keras_nlp
import keras_nlp
import keras

preset = "distil_bert_base_en_uncased"

preprocessor = keras_nlp.models.DistilBertPreprocessor.from_preset(
    preset,
    sequence_length=160,
    name="tweet_preprocessor"
)

classifier = keras_nlp.models.DistilBertClassifier.from_preset(
    preset,
    preprocessor=preprocessor,
    num_classes=2
)

classifier.summary()

classifier.compile(
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=keras.optimizers.Adam(1e-5),
    metrics=["accuracy"]
)

history = classifier.fit(
    x=X_train,
    y=y_train,
    batch_size=32,
    epochs=2,
    validation_data=(X_val, y_val)
)

Preprocessor: "tweet_preprocessor"
```

Layer (type)	
Config	

distil_bert_tokenizer (DistilBertTokenizer)
Vocab size: 30,522

Model: "distil_bert_text_classifier"

Layer (type)	Output Shape
--------------	--------------

Param #	Connected to
---------	--------------

padding_mask (InputLayer)	(None, None)
---------------------------	--------------

0	-
---	---

token_ids (InputLayer)	(None, None)
------------------------	--------------

0	-
---	---

distil_bert_backbone	(None, None, 768)
----------------------	-------------------

66,362,880	padding_mask[0][0], (DistilBertBackbone) token_ids[0][0]
------------	--

get_item (GetItem)	(None, 768)
--------------------	-------------

0	distil_bert_backbone[0][0]
---	----------------------------

pooled_dense (Dense)	(None, 768)
----------------------	-------------

590,592	get_item[0][0]
---------	----------------

output_dropout (Dropout)	(None, 768)
--------------------------	-------------

0	pooled_dense[0][0]
---	--------------------

logits (Dense)	(None, 2)
----------------	-----------

1,538	output_dropout[0][0]
-------	----------------------

Total params: 66,955,010 (255.41 MB)

Trainable params: 66,955,010 (255.41 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/2

191/191 ————— 3658s 19s/step - accuracy: 0.6947 - loss: 0.5734 - val_accuracy: 0.8444 - val_loss: 0.3940

Epoch 2/2

191/191 ————— 3615s 19s/step - accuracy: 0.8412 - loss: 0.3806 - val_accuracy: 0.8470 - val_loss: 0.3813

#loading the sample_submission.csv file into a pandas DataFrame

sample_submission

#to use as a template for submission

sample_submission =

pd.read_csv("/kaggle/input/nlp-getting-started/sample_submission.csv")

#predicting the class labels for my test data (X_test) using my trained classifier, then

#getting the index of the class with the highest probability using np.argmax, and finally

#assigning these predicted labels to the "target" column of my sample_submission DataFrame

sample_submission["target"] = np.argmax(classifier.predict(X_test), axis=1)

102/102 ————— 481s 5s/step

#Printing first five entries

sample_submission.head()

	id	target
0	0	1
1	2	1
2	3	1
3	9	1
4	11	1

#Submitting in .csv format for evaluation

sample_submission.to_csv("submission.csv", index=False)

Conclusion:

Using DistilBERT model for this NLP classification problem an accuracy of 0.83573 has been achieved on Test Dataset. More models should be explored to improve the accuracy value.